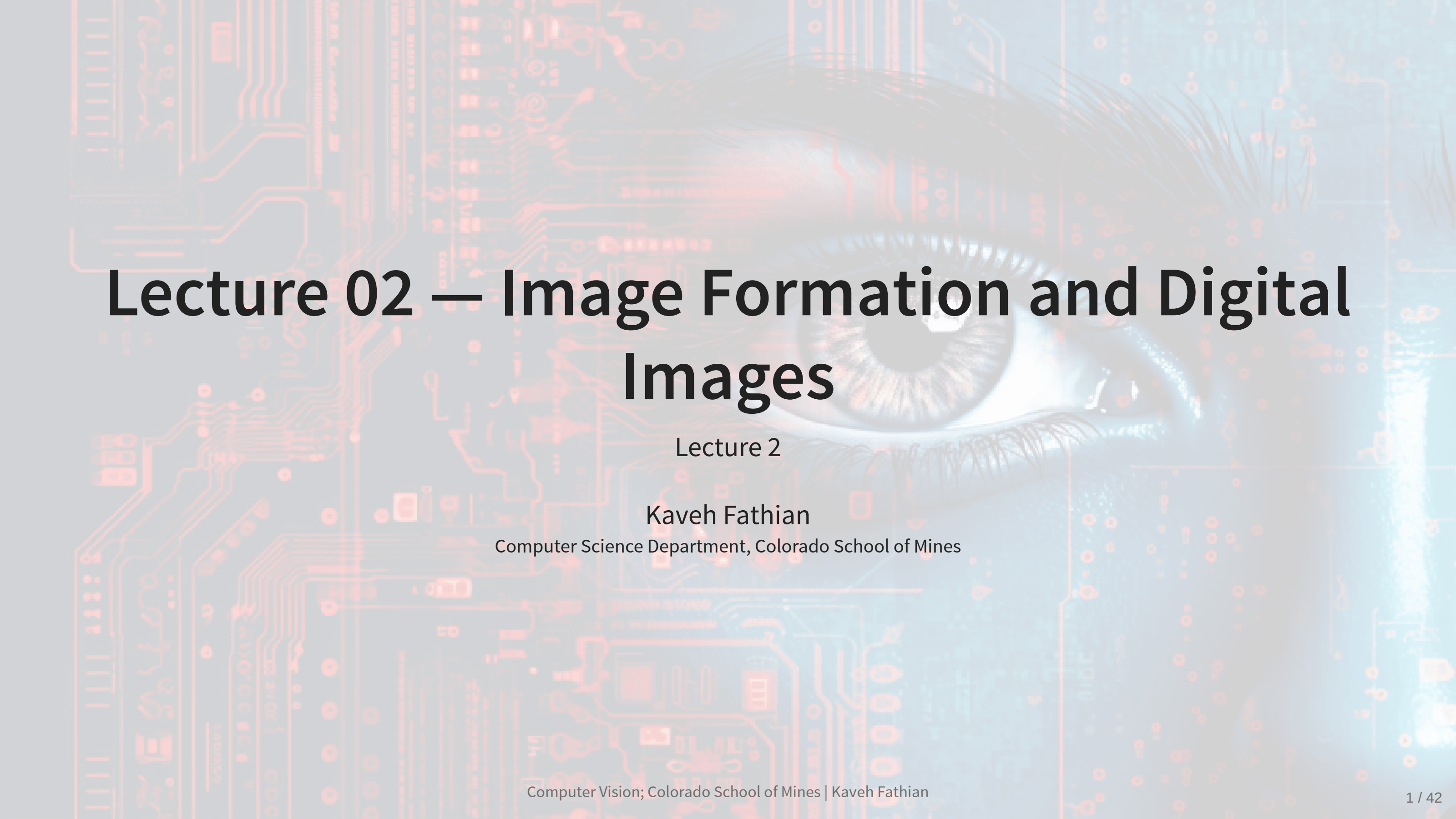




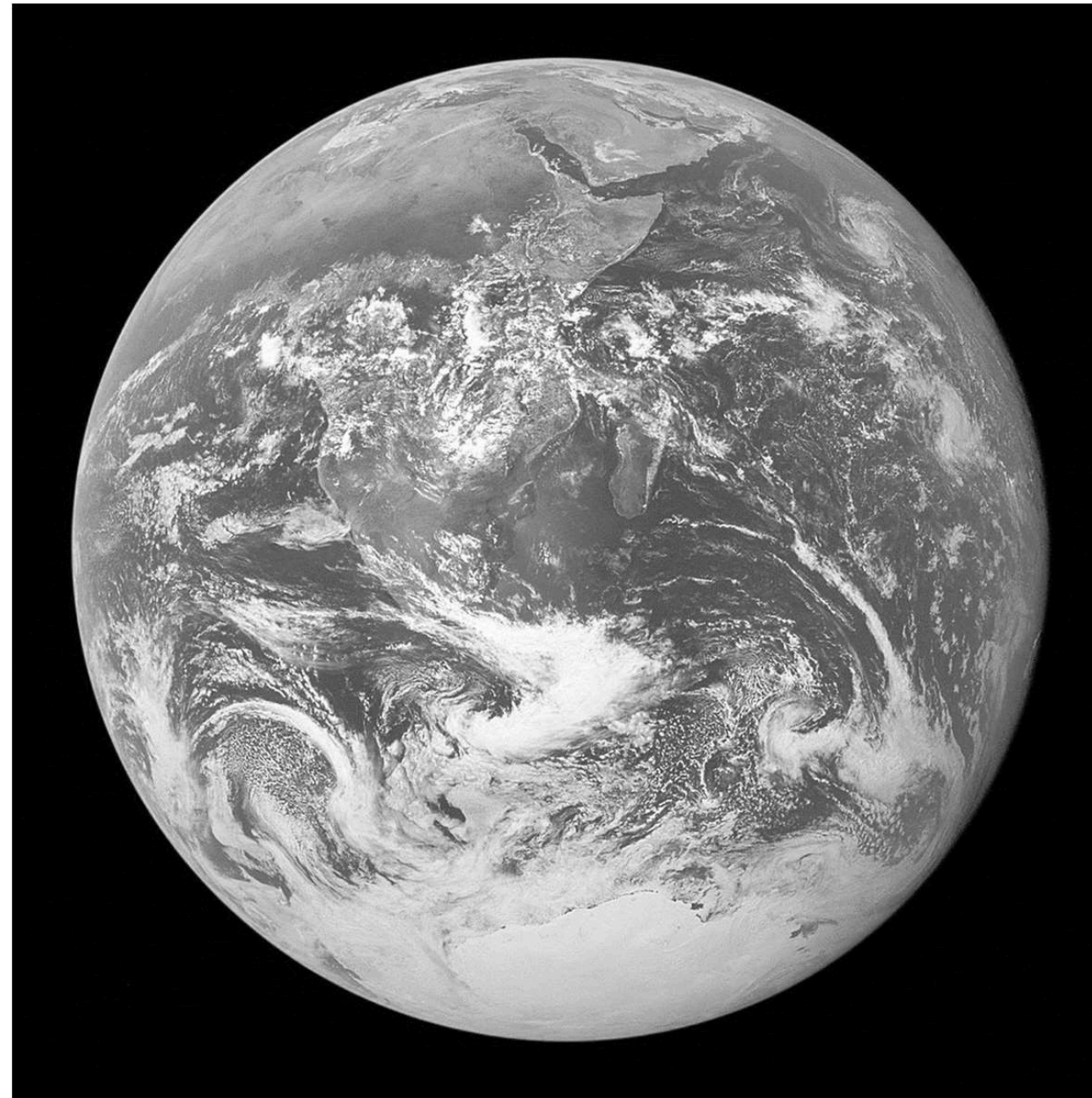
# Lecture 02 — Image Formation and Digital Images

Lecture 2

Kaveh Fathian

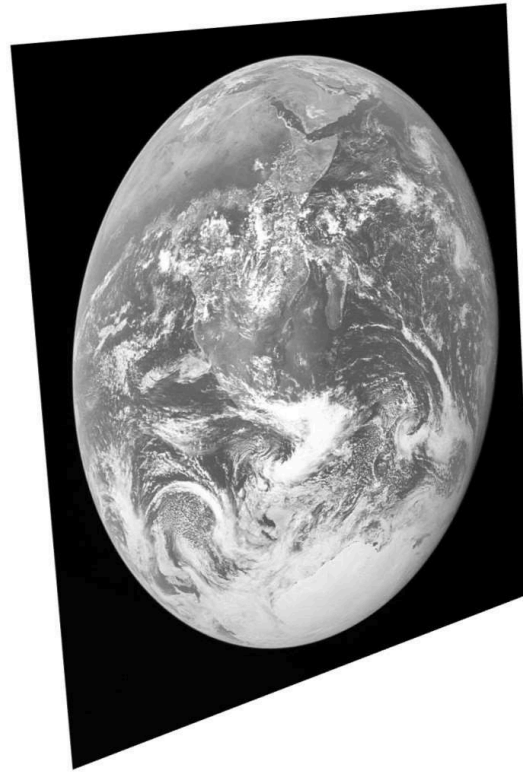
Computer Science Department, Colorado School of Mines

# What is an Image and how is it formed?

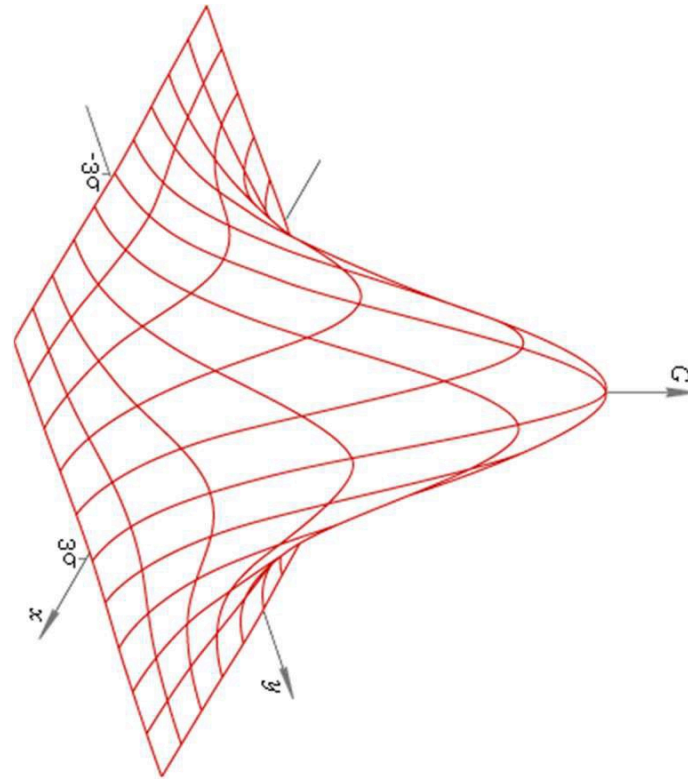


Blue Marble, NASA / Apollo 17

# Image Formation



Scene / object



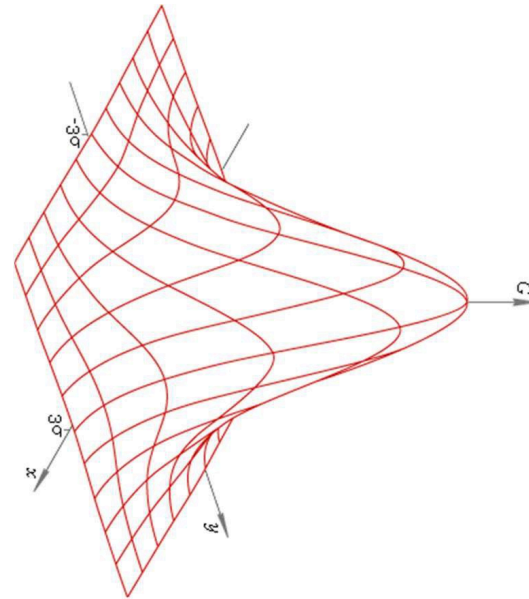
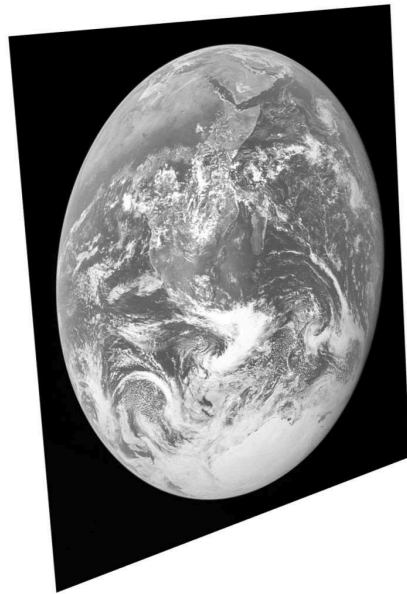
Light as a signal



Camera / sensor

A camera measures light from the world and turns it into a digital image.

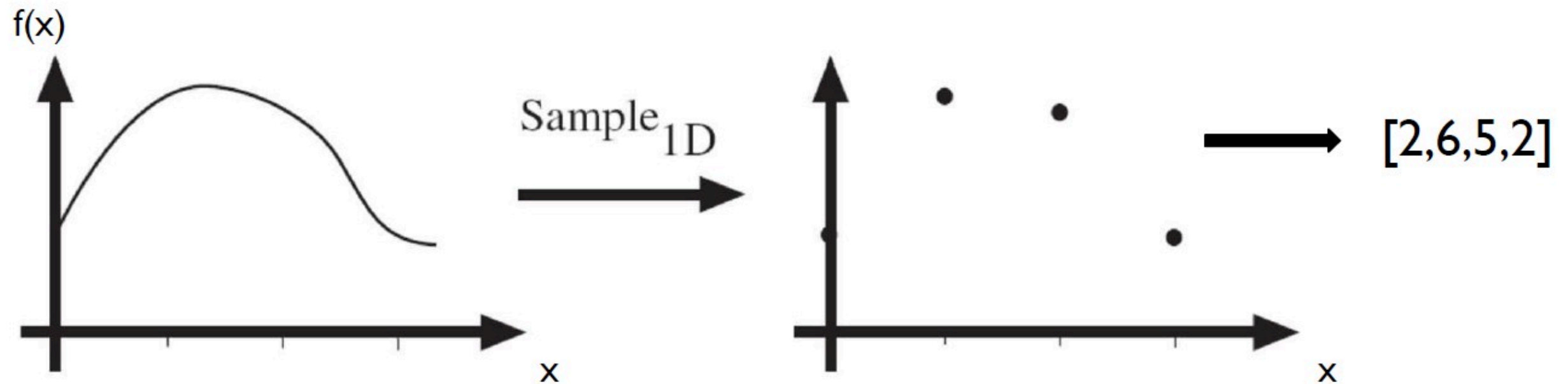
# Signal



**Signal:** A (multi-dimensional) function that contains information about a phenomenon (e.g., light, heat, gravity, sound, pressure, motion).

- Signals can be **continuous**, such as light in the physical world.
- Measurements are often **discrete** samples of that continuous signal.
- Sampling reduces a continuous signal to a finite set of values.

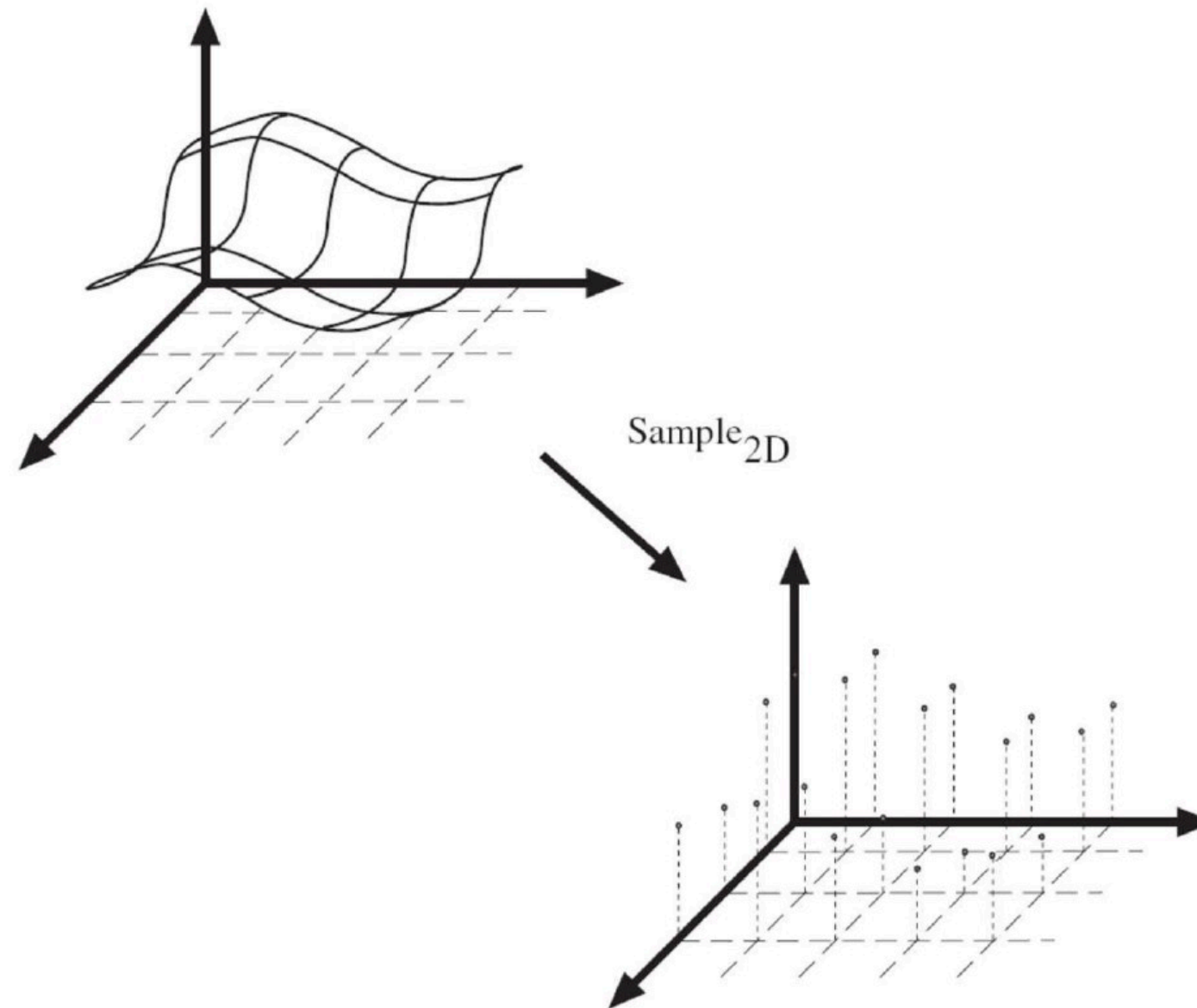
# Sampling in 1D



- Sampling a 1D continuous function returns a vector whose elements are values of that function at selected sample points.
- Instead of the full function, we store values at selected locations.

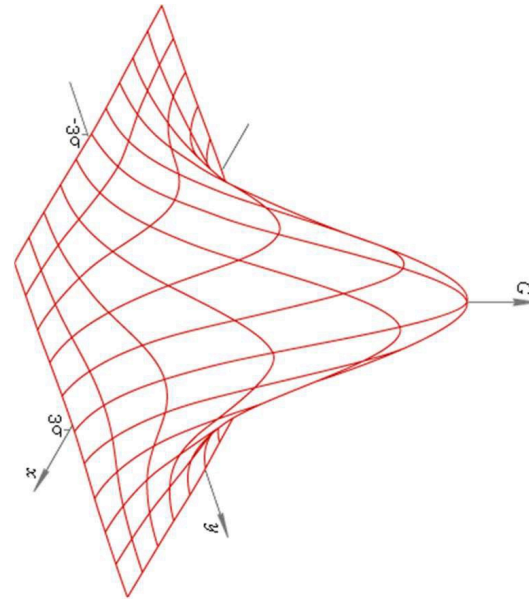
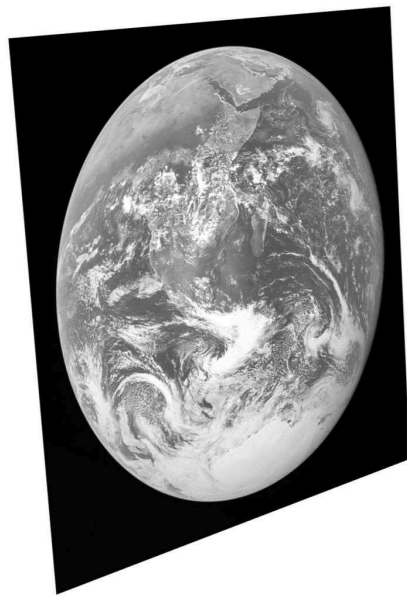


# Sampling in 2D



Sampling a 2D function returns a matrix.

# 2D Image



**Image:** A sampling of a function that contains information about a 2D signal.

- The function stores **brightness** or **intensity**.
- The image samples brightness along the  $x$  and  $y$  dimensions.
- A video can be viewed as a time-varying 2D image signal:  $(x, y, t)$ .

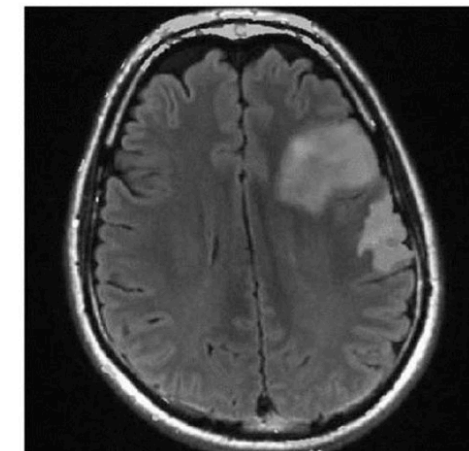
## *i* Note

Cameras observe a 2D projection of the 3D world.

# Examples of 2D Images

2D images appear in many sensing modalities:

- Natural images
- Thermal images
- Satellite/aerial images
- Microscopy
- Ultrasound
- MRI / CT slices
- Radar images
- Depth images

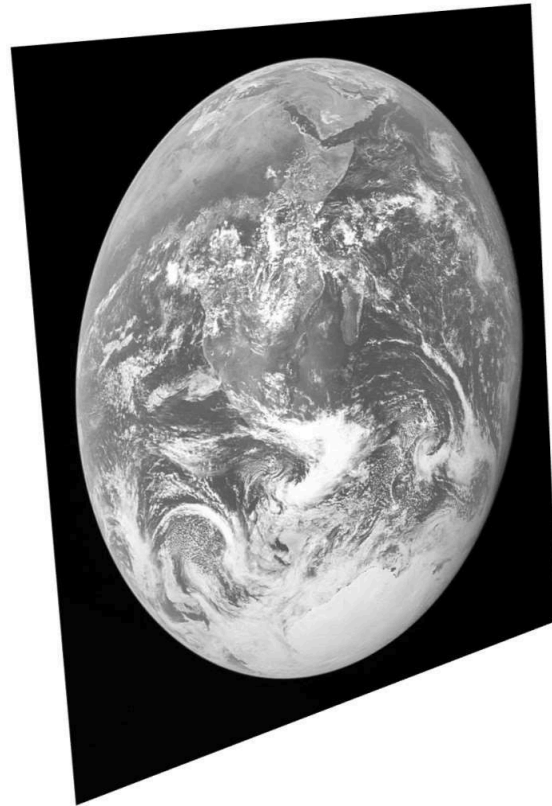


## *i* Note

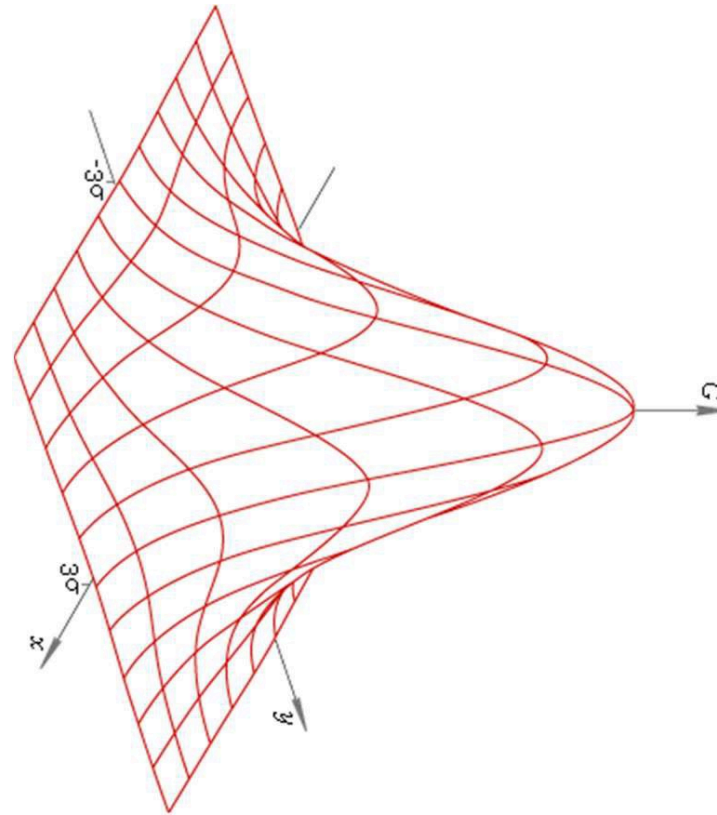
The signal may **not** always be visible light. An image arranges what we have measured over a 2D domain.



# Sampling in Practice



World / scene



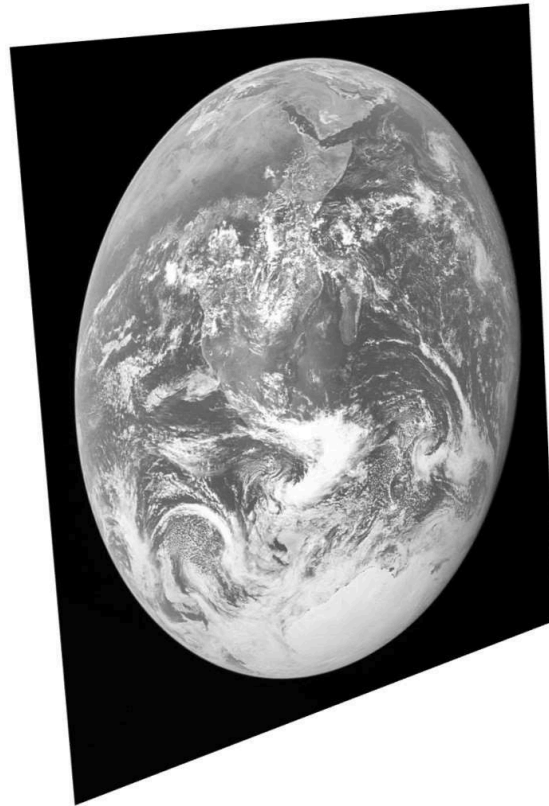
Continuous signal



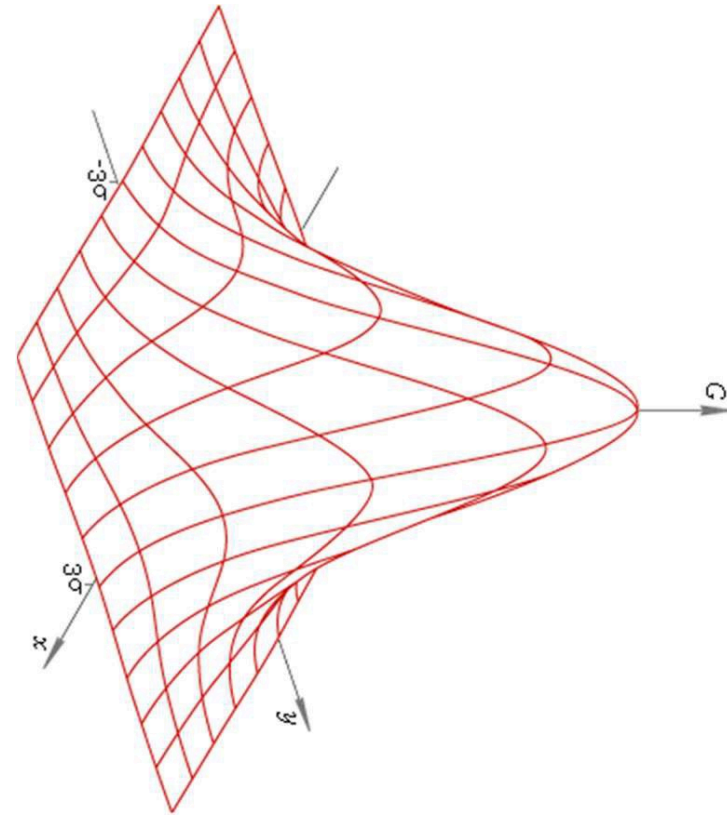
Camera

The imaging pipeline converts a continuous physical signal into discrete sensor measurements.

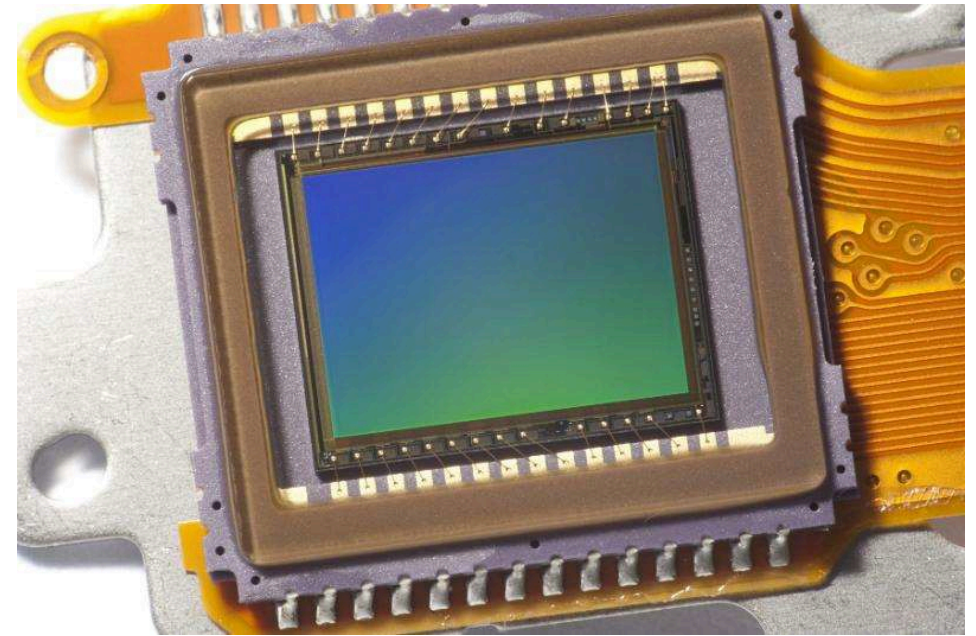
# Sampling in Practice: Digital Image



Scene / image plane



Light signal

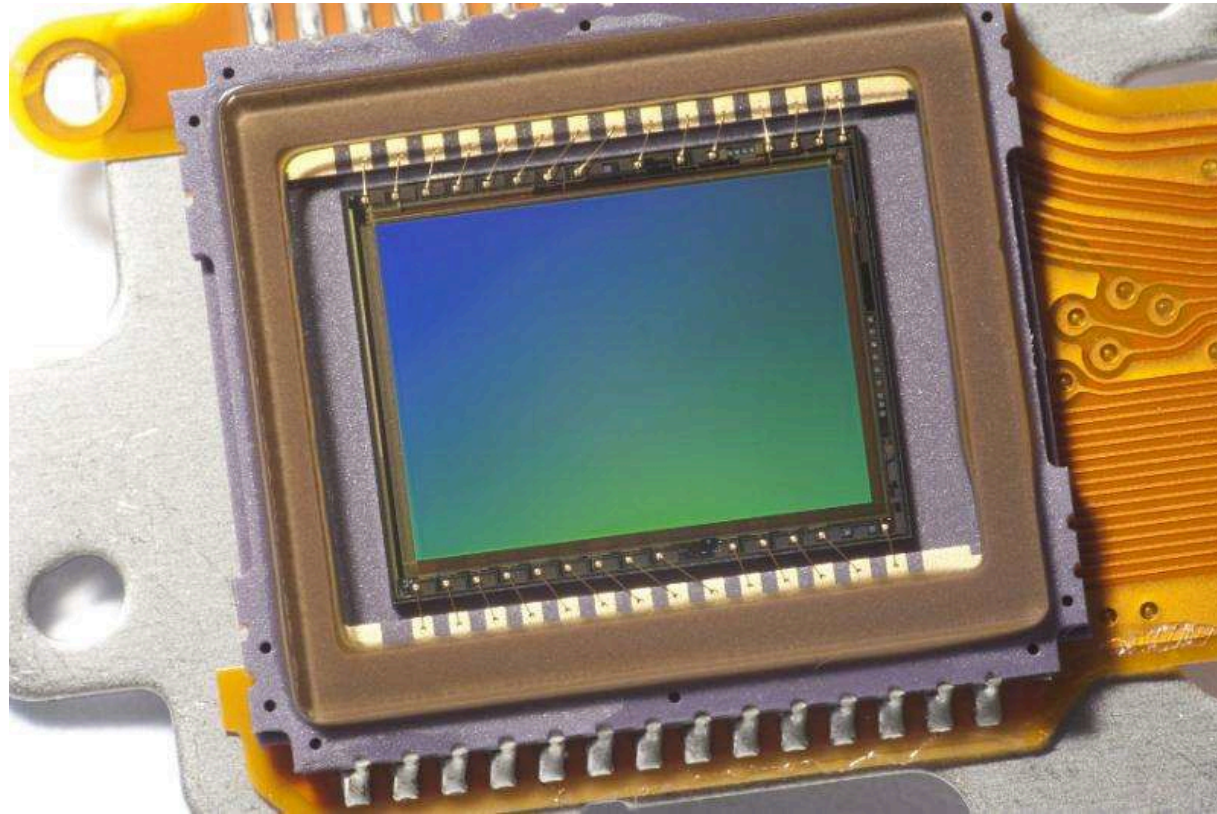


CMOS / CCD sensor

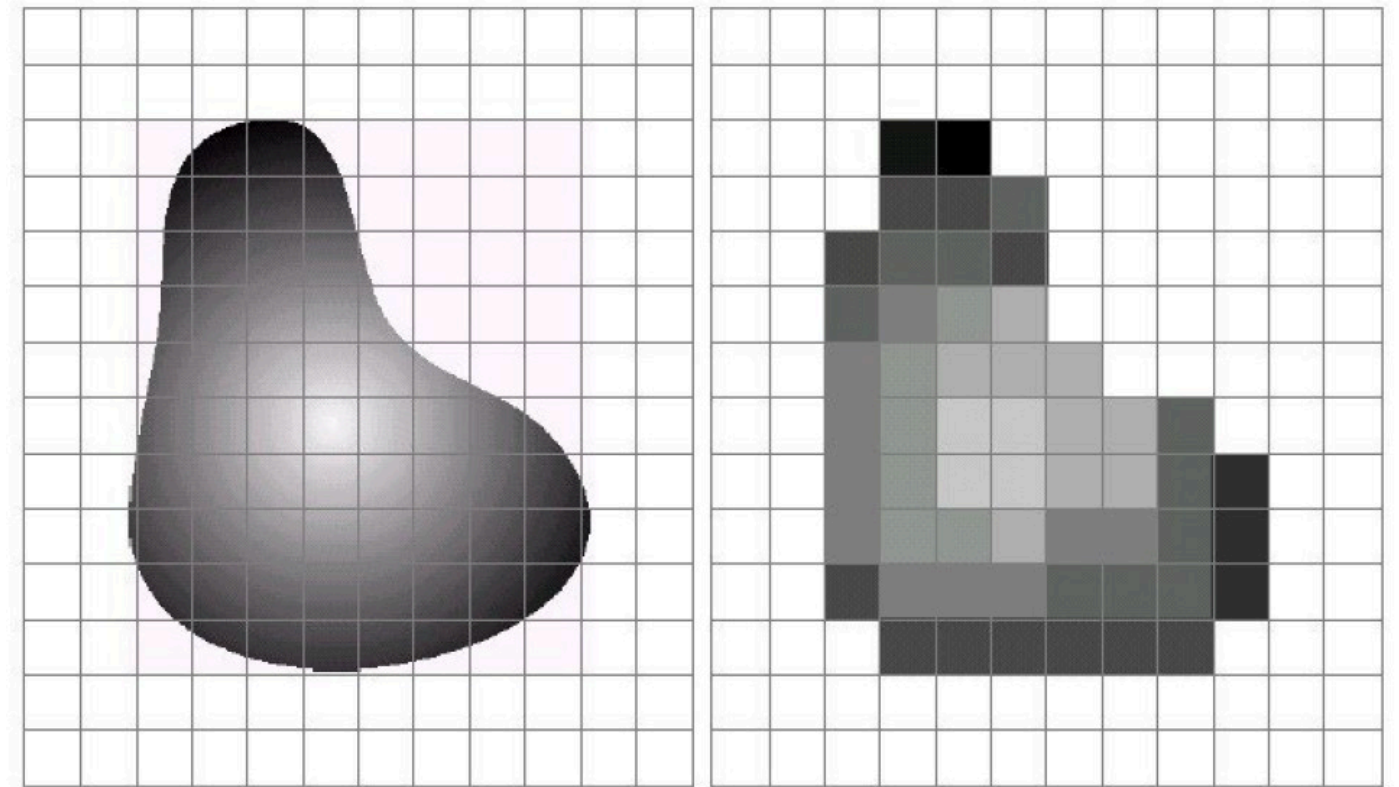
Digital cameras use sensor arrays to sample light and produce a grid of pixel values.



# Sensor Array



Camera / sensor hardware



CMOS sensor array

Continuous light is **sampled** and **quantized** by the 2D sensor array (pixels) to produce a digital image.

# Elements of a Digital Image

A pixel is a **picture element**.

For a grayscale image:

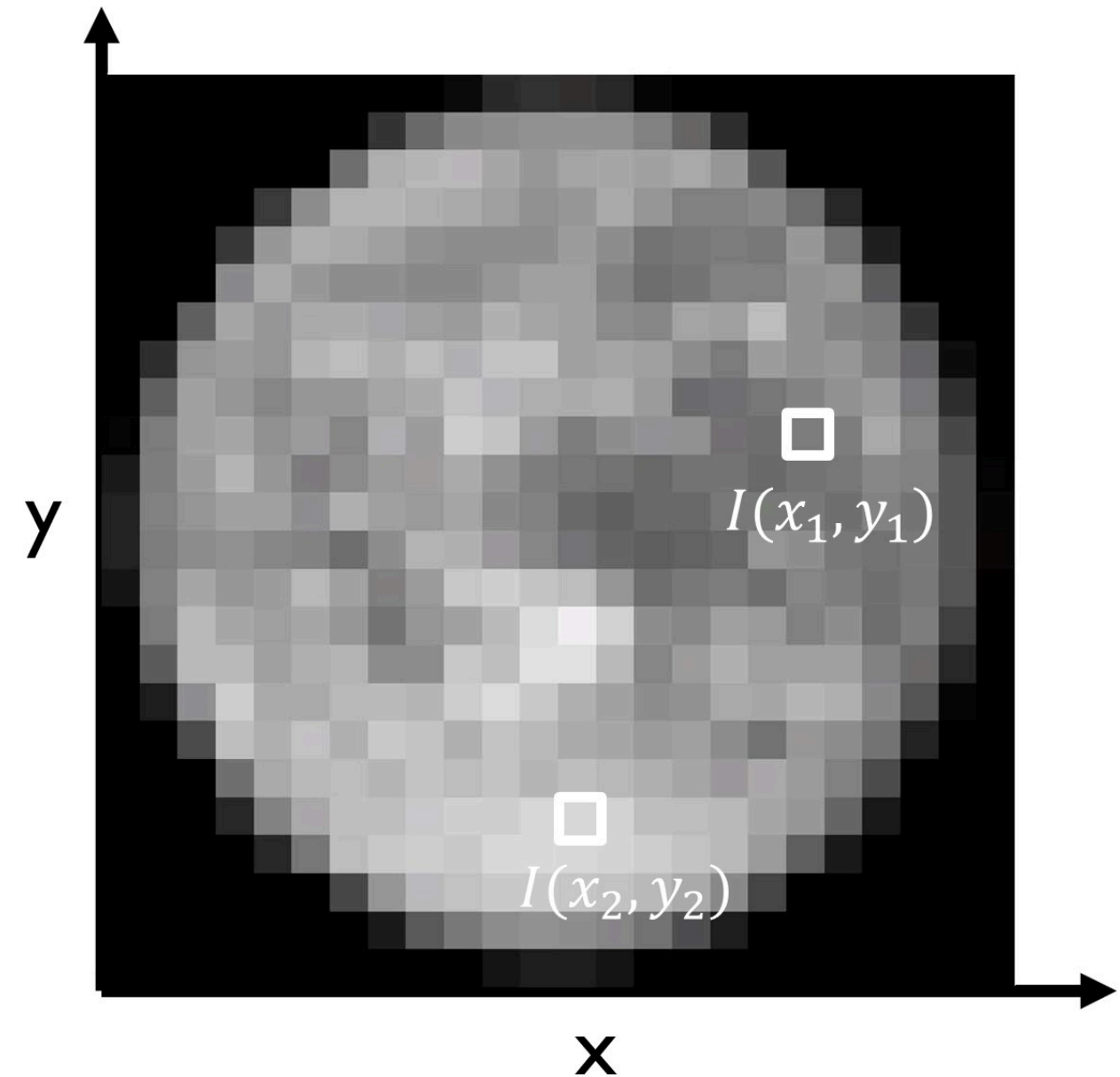
$$I(x, y) = \text{intensity at image location}(x, y)$$

- $x$ : horizontal coordinate
- $y$ : vertical coordinate

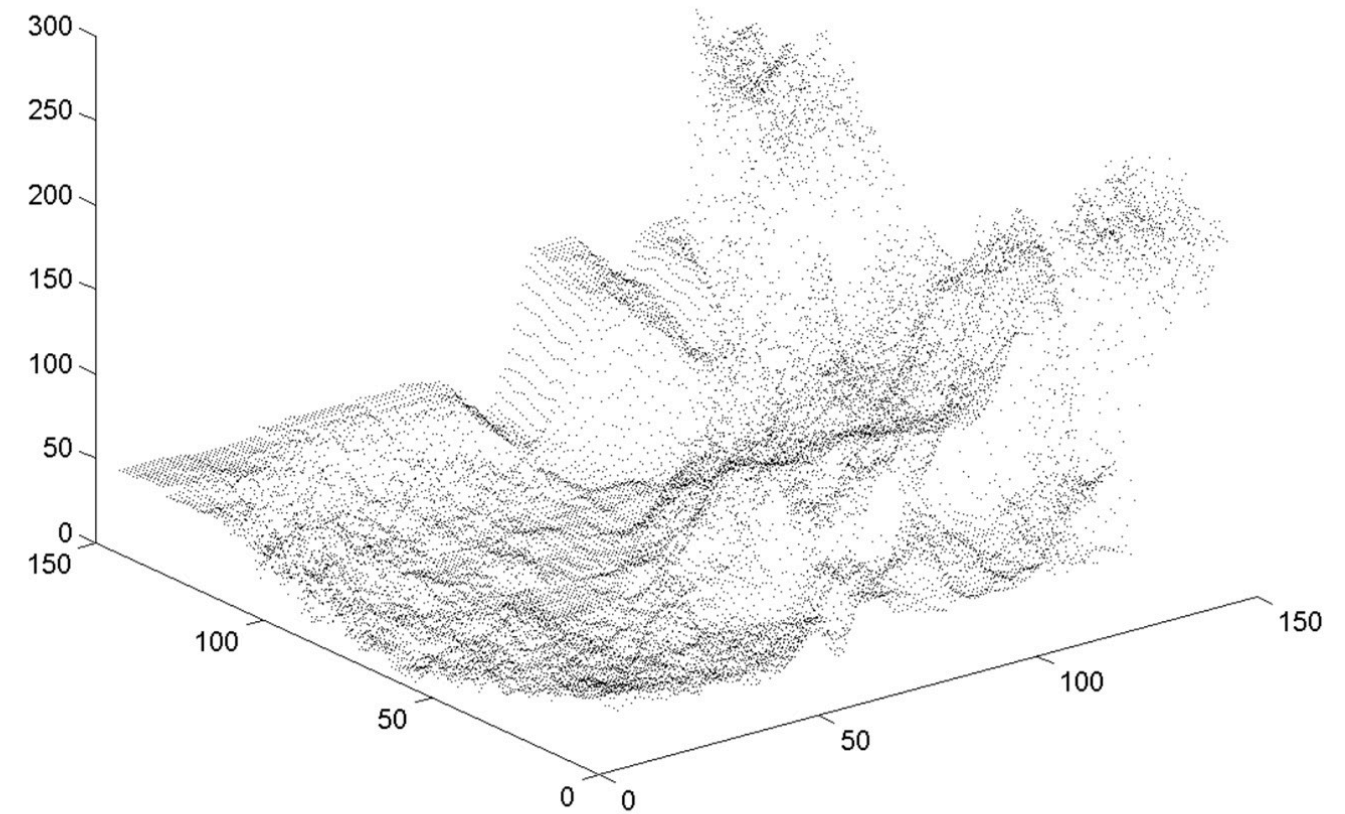
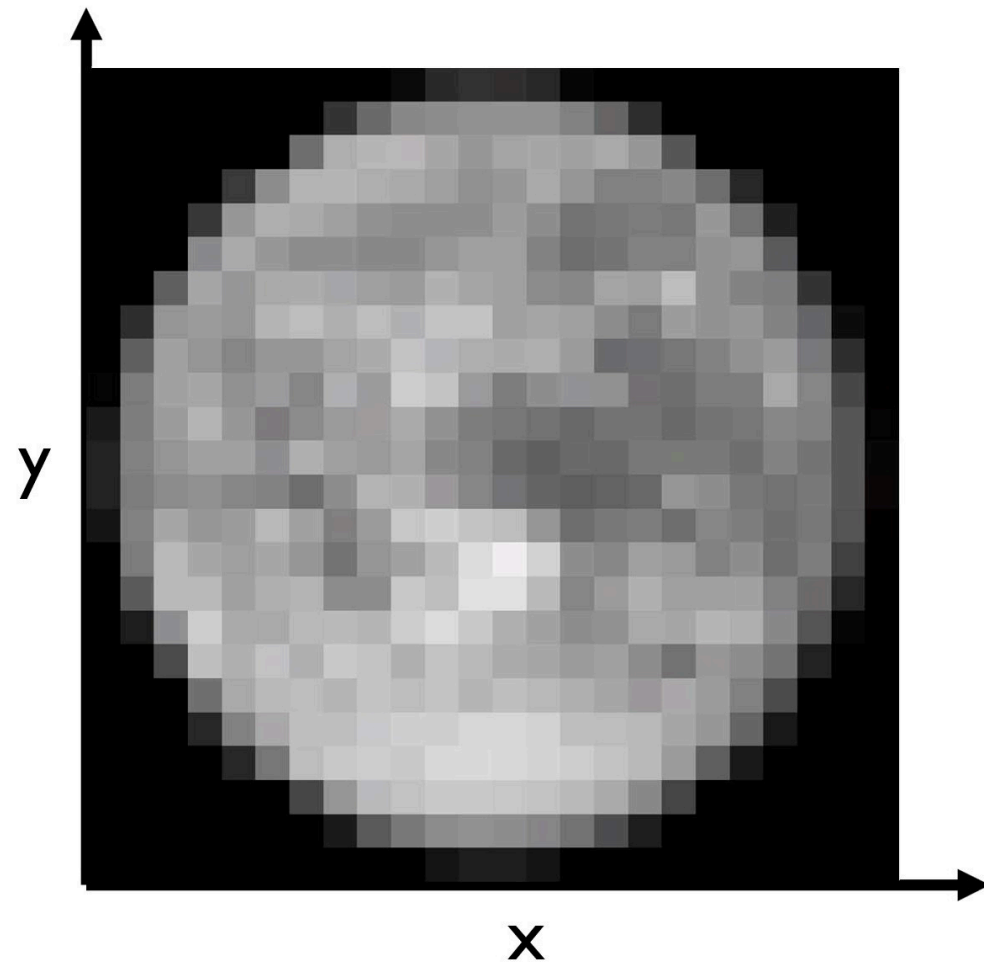
**Example:**

```
1 I(x1, y1) = 101  
2 I(x2, y2) = 191
```

Different pixel locations can have different intensity values.



# Digital Image is a 2D Signal

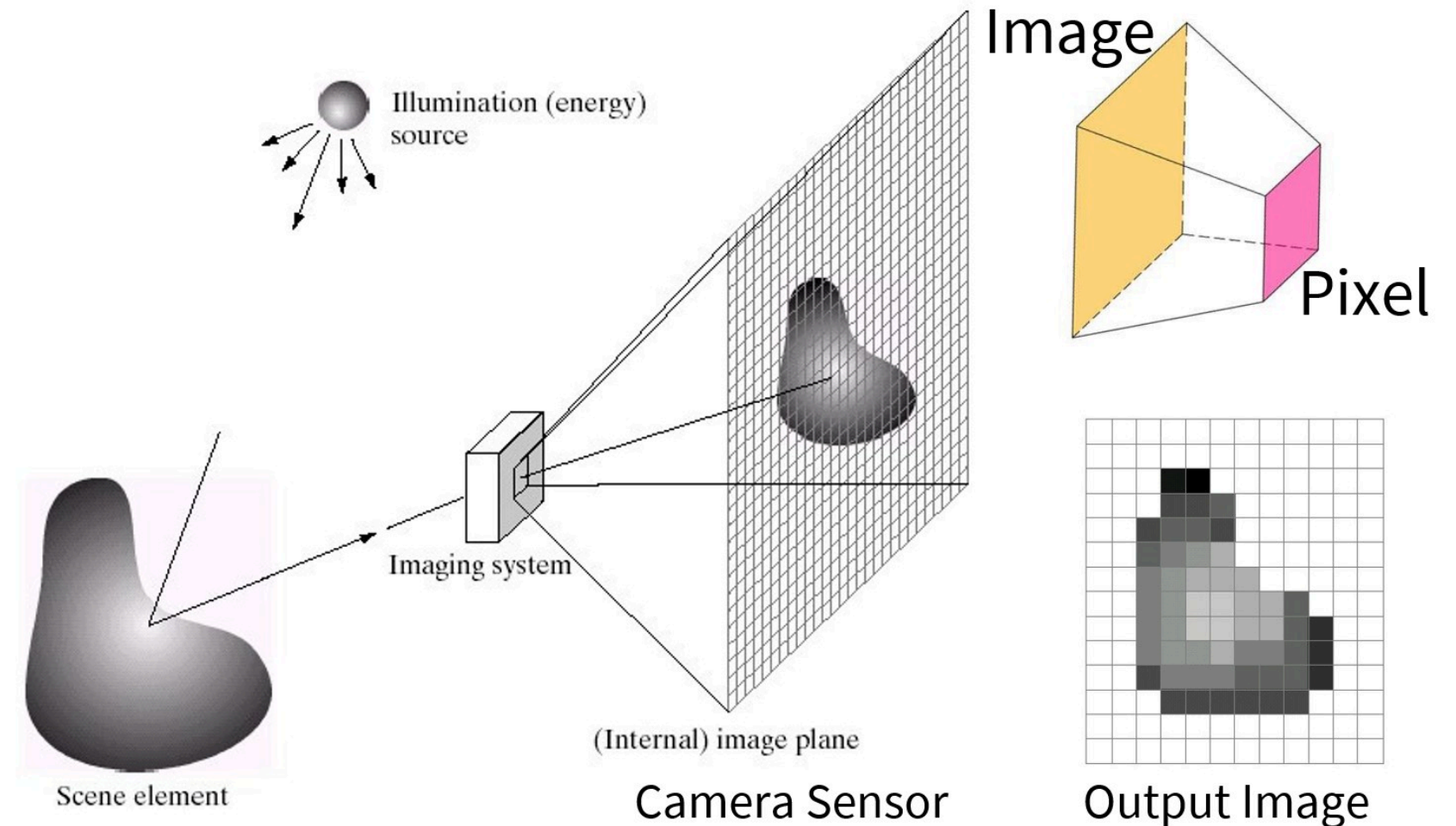


A digital image can be interpreted as a 2D function whose value is **brightness** or **intensity**.



# Light Integration Over a Pixel Region

- A single pixel does not measure an infinitely thin ray.
- It measures light integrated over a small region of the sensor and the corresponding cone/frustum of incoming light.

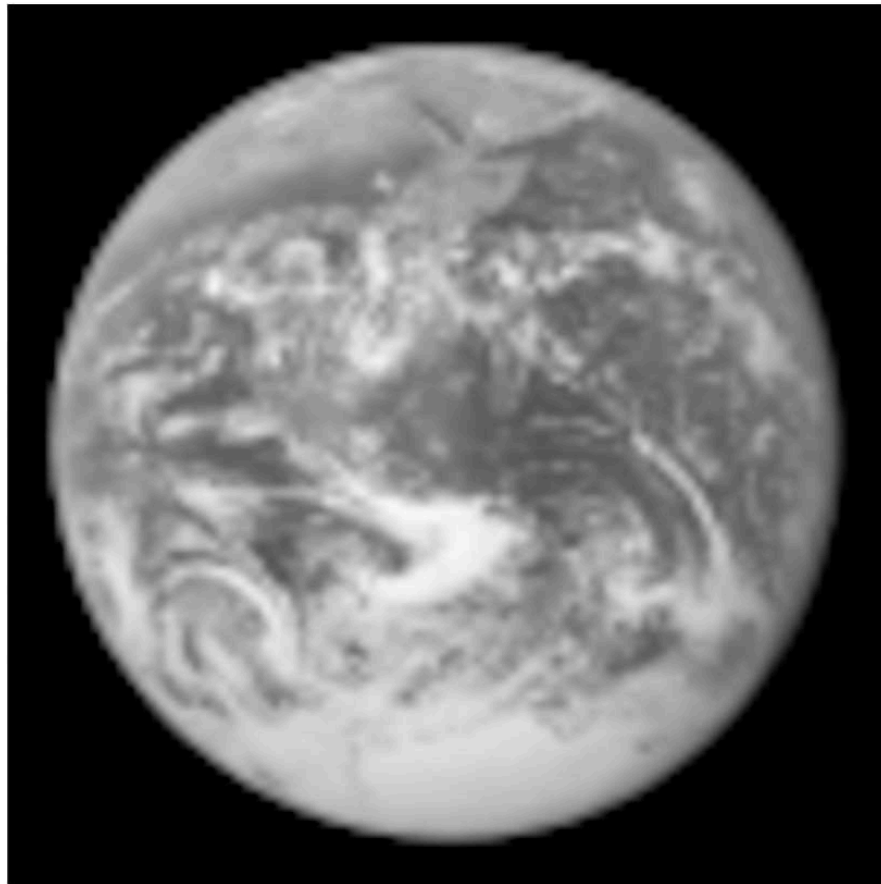


## *i* Note

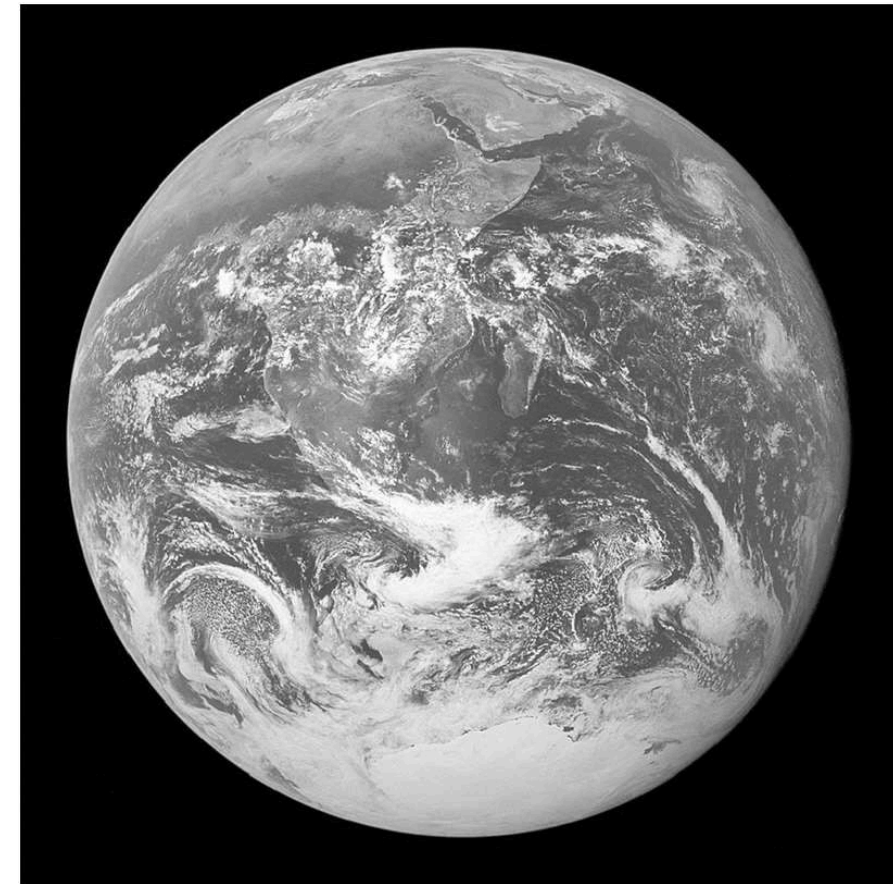
This is one reason why cameras blur, average, and alias details that are smaller than a pixel can represent.

# Resolution: Geometric vs. Spatial

Both images can have the same number of pixels, but the amount of **scene detail** represented can be different.



Low geometric resolution

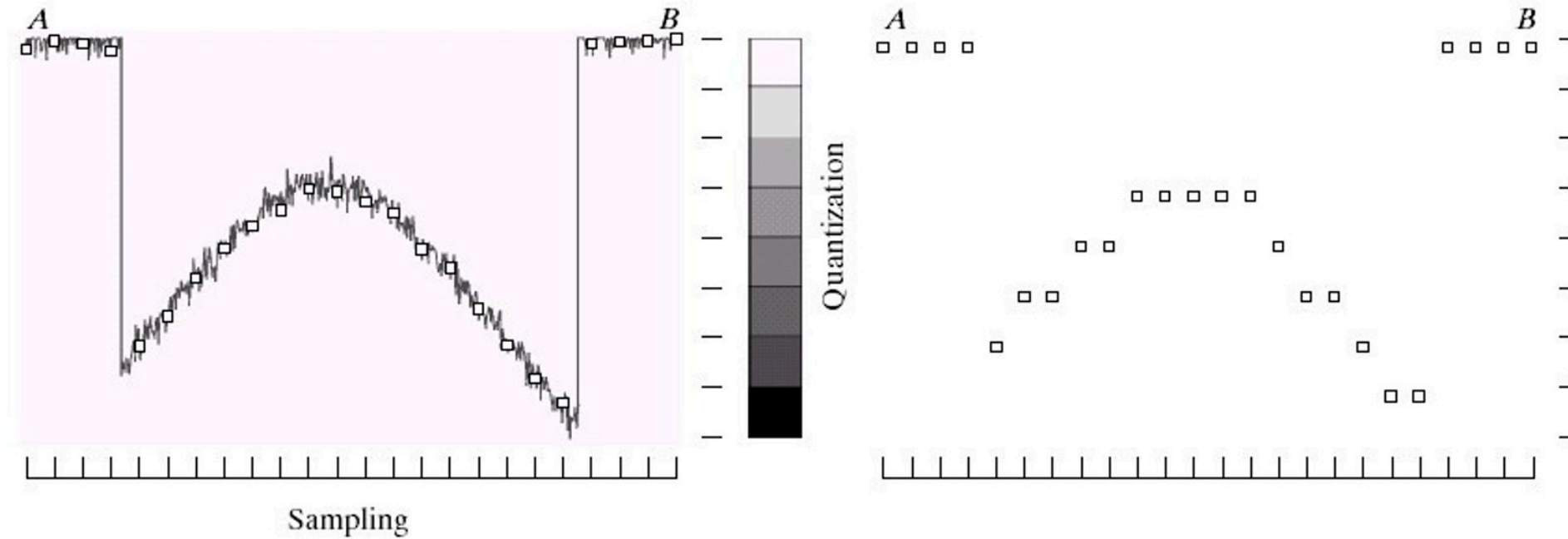


High geometric resolution

## *i* Note

Spatial resolution counts pixels. Geometric resolution is how accurately those pixels represent the positions of objects in the real world.

# Quantization

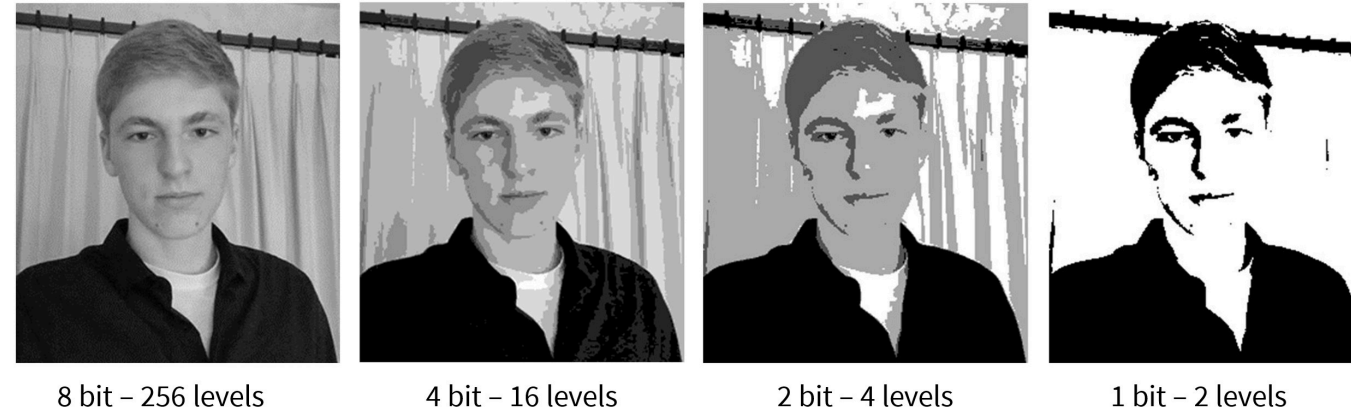


- Sampling chooses **where** to measure a signal.
- Quantization chooses **which values** can be represented.
- For example, an 3 bits can represent:

$$2^3 = 8$$

possible intensity levels in a grayscale image.

# Quantization Effects — Radiometric Resolution



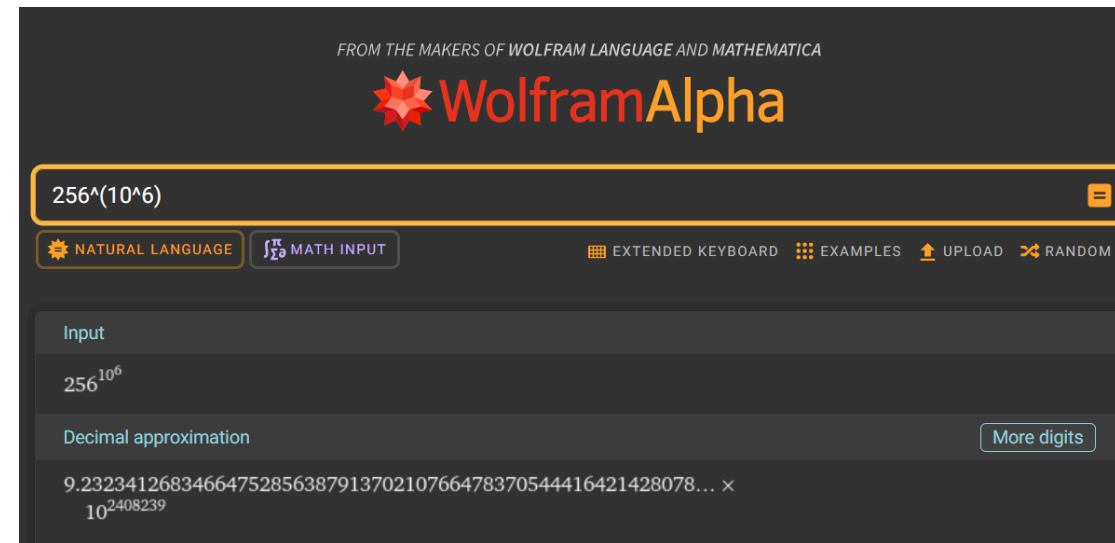
- Radiometric resolution is the number of distinct intensity levels available.

| Bit depth | Number of levels |
|-----------|------------------|
| 1 bit     | 2 levels         |
| 2 bit     | 4 levels         |
| 4 bit     | 16 levels        |
| 8 bit     | 256 levels       |

- Higher bit depth can represent more subtle intensity differences.
- In photography, this is closely related to dynamic range.



# Dimensionality of an Image



- An image of size  $1000 \times 1000$  with 8-bit quantization per pixel has:

$$256^{1000 \times 1000} \approx 10^{2,408,239}$$

possible intensity configurations.

- This is an enormous high-dimensional space – the known universe has only about  $10^{80}$  atoms!
- Computer vision works in this extremely high dimensional space – but real images are not arbitrary arrays and they occupy a small, structured subset of this space.

# Images in Python: Grayscale Arrays

For an  $N \times M$  grayscale image `im`:

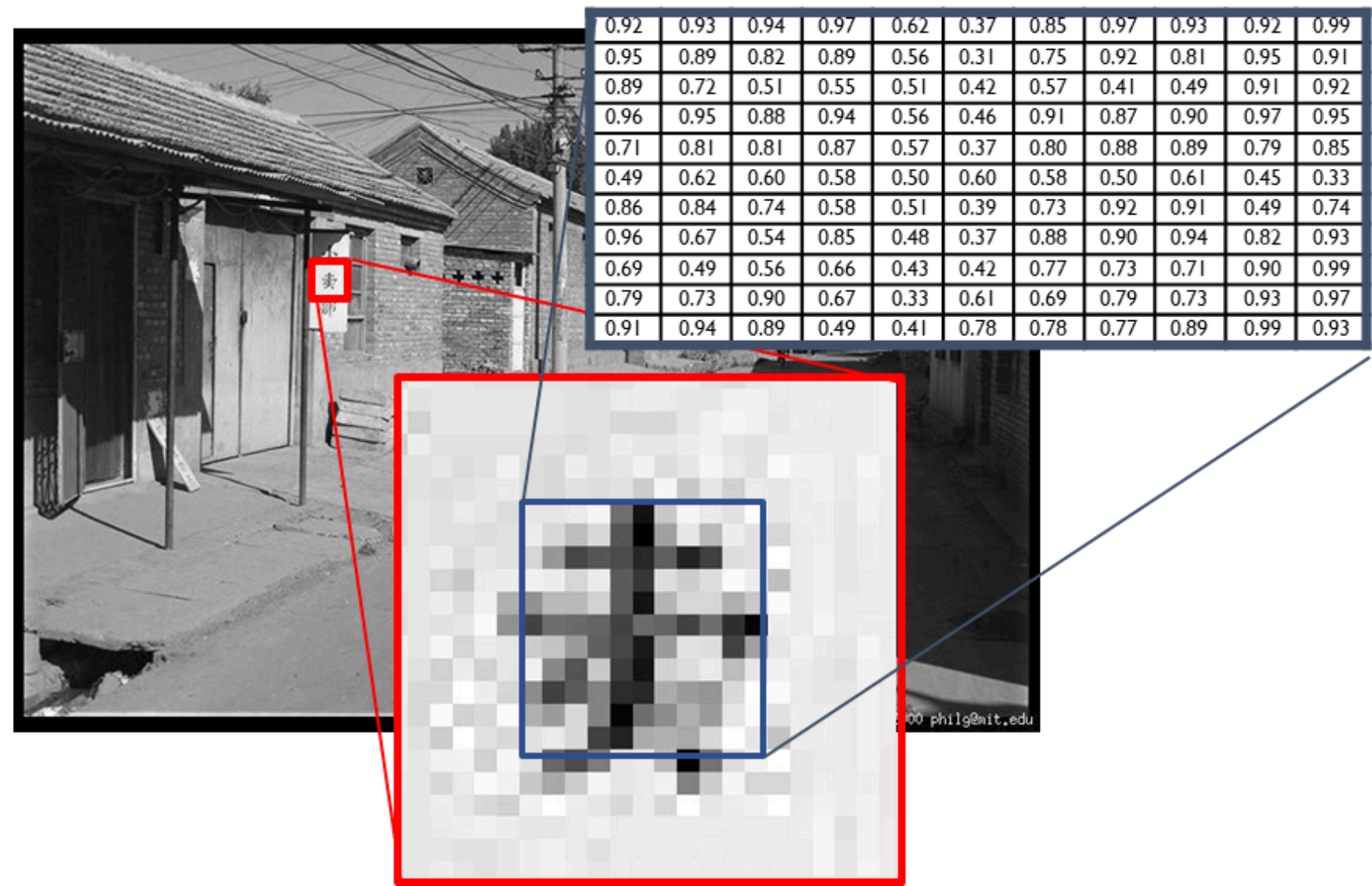
```
1 im[0, 0]          # top-left pixel value
2 im[y, x]          # y rows down, x columns right
3 im[N-1, M-1]      # bottom-right pixel value
```

|      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|
| 0.92 | 0.93 | 0.94 | 0.97 | 0.62 | 0.37 | 0.85 | 0.97 | 0.93 | 0.92 | 0.99 |
| 0.95 | 0.89 | 0.82 | 0.89 | 0.56 | 0.31 | 0.75 | 0.92 | 0.81 | 0.95 | 0.91 |
| 0.89 | 0.72 | 0.51 | 0.55 | 0.51 | 0.42 | 0.57 | 0.41 | 0.49 | 0.91 | 0.92 |
| 0.96 | 0.95 | 0.88 | 0.94 | 0.56 | 0.46 | 0.91 | 0.87 | 0.90 | 0.97 | 0.95 |
| 0.71 | 0.81 | 0.81 | 0.87 | 0.57 | 0.37 | 0.80 | 0.88 | 0.89 | 0.79 | 0.85 |
| 0.49 | 0.62 | 0.60 | 0.58 | 0.50 | 0.60 | 0.58 | 0.50 | 0.61 | 0.45 | 0.33 |
| 0.86 | 0.84 | 0.74 | 0.58 | 0.51 | 0.39 | 0.73 | 0.92 | 0.91 | 0.49 | 0.74 |
| 0.96 | 0.67 | 0.54 | 0.85 | 0.48 | 0.37 | 0.88 | 0.90 | 0.94 | 0.82 | 0.93 |
| 0.69 | 0.49 | 0.56 | 0.66 | 0.43 | 0.42 | 0.77 | 0.73 | 0.71 | 0.90 | 0.99 |
| 0.79 | 0.73 | 0.90 | 0.67 | 0.33 | 0.61 | 0.69 | 0.79 | 0.73 | 0.93 | 0.97 |
| 0.91 | 0.94 | 0.89 | 0.49 | 0.41 | 0.78 | 0.78 | 0.77 | 0.89 | 0.99 | 0.93 |

## ⚠ Important

In NumPy, image indexing is usually **row first, column second**: `im[y, x]`, not `im[x, y]`.

# Grayscale Intensity



A grayscale image is a **matrix** that stores one intensity value per pixel.

# Color Images

- A color image stores multiple intensity values per pixel.
- For RGB images, each pixel has three channels:
  - Red intensity
  - Green intensity
  - Blue intensity

1 `image shape: height × width × 3`



## *i* Note

Different libraries may use different channel order conventions. OpenCV commonly uses BGR; Matplotlib commonly expects RGB.



# Images in Python: Color Arrays

For an  $N \times M$  color image `im` with three channels:

```
1 im[0, 0, 0]      # top-left pixel, first channel
2 im[y, x, 1]      # pixel at row y, column x, second channel
3 im[N-1, M-1, 2]  # bottom-right pixel, third channel
```

Typical shape:

```
1 im.shape # (height, width, channels)
```

For an RGB image:

```
1 channel 0 = red
2 channel 1 = green
3 channel 2 = blue
```

For OpenCV-loaded images:

```
1 channel 0 = blue
2 channel 1 = green
3 channel 2 = red
```

# Images in Python: Data Types

Take care of image data types.

```
1 # uint8 image: values 0 to 255
2 im = cv2.imread("file.jpg")
3
4 # convert to float32, still values 0 to 255
5 im_float = im.astype(np.float32)
6
7 # normalize to 0 to 1
8 im_norm = im.astype(np.float32) / 255.0
```

## ⚠ Important

Many image-processing bugs come from mixing `uint8`, `float32`, and different value ranges.

# Random Images in Python

```
1 from numpy import random as r
2
3 I = r.rand(256, 256)
```

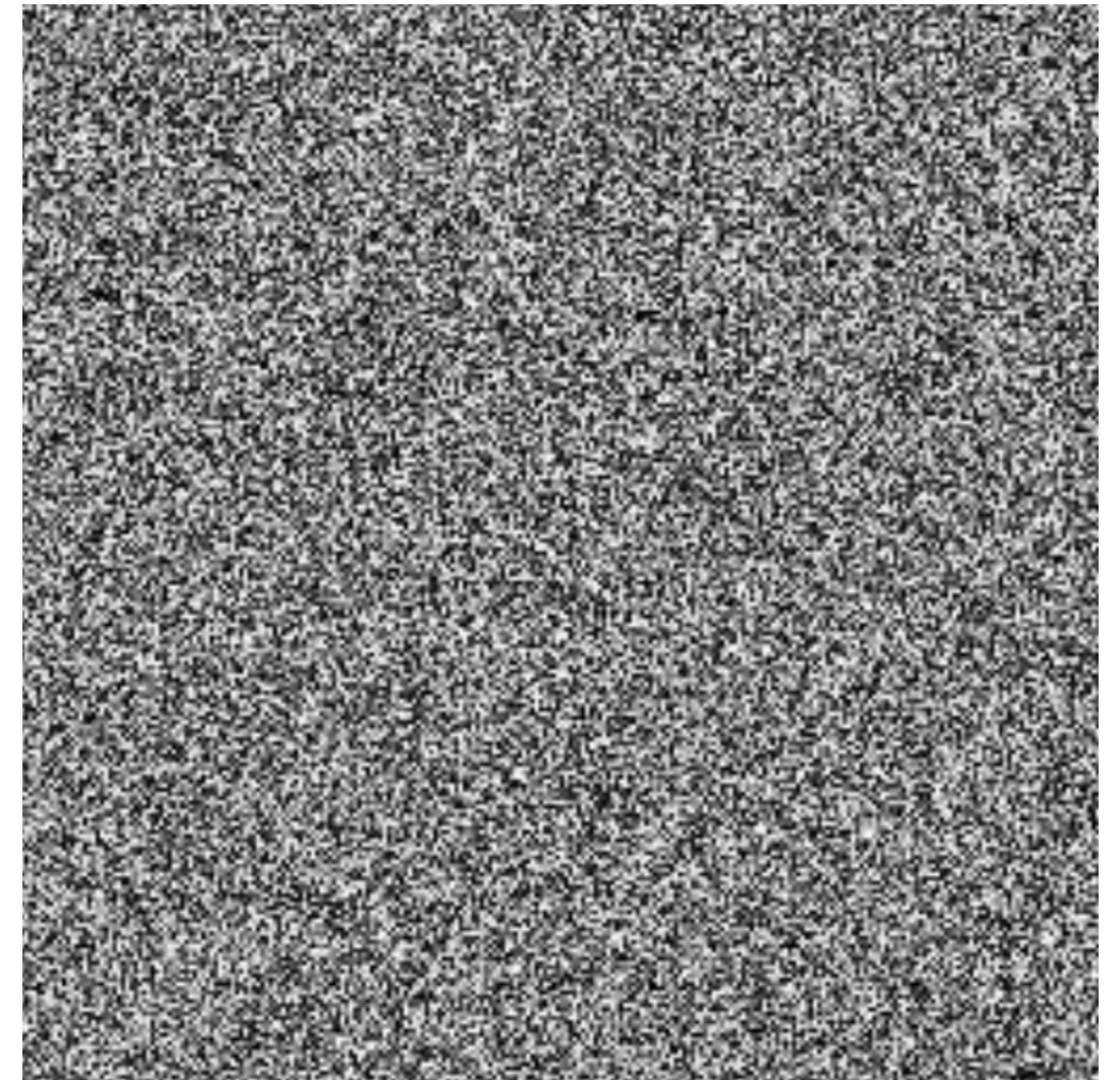
`r.rand(256, 256)` creates a 256 by 256 array of random floating-point values in the range  $[0, 1)$ .

## Questions:

- What is `I`?
- What does it look like?
- Which values does it contain?
- How many values are there?

# Displaying an Image in Python

```
1 from matplotlib import pyplot as plt
2 from numpy import random as r
3
4 I = r.rand(256, 256)
5
6 plt.imshow(I, cmap="gray")
7 plt.axis("off")
8 plt.show()
```



Is this an image?



# Coding in This Course

We will use:

- **Python** for programming
- **Conda** for managing the course environment
- **NumPy** for numerical arrays
- **OpenCV** for image processing and computer vision
- **PyTorch** for deep learning topics
- **VS Code** for editing, debugging, and notebooks

## ⚠ Important

Do not use GenAI to solve homework for you. If you copy-paste code without understanding it, you will not learn how to code.

Be kind and supportive: students come from different backgrounds and have different levels of Python experience.

# Python / Tool Refresher

Course tutorials are available here:

<https://ariarobotics.github.io/cv/tutorials/>

Review as needed:

- **WSL + Ubuntu** — recommended Linux environment for Windows users
- **Git & GitHub** — cloning, committing, pulling, pushing, and GitHub Classroom
- **Python & Conda** — installing Conda and creating the course environment
- **Visual Studio Code** — Python interpreter, notebooks, and debugging
- **NumPy** — arrays, indexing, vectorization, matrices, and image-related operations



Tip

These tutorials are reference material. Use them when you need help with the tools so class time can focus on computer vision.

# Recommended Development Setup

## Windows

Recommended workflow:

```
1 Windows
2   ↓
3 WSL 2 + Ubuntu
4   ↓
5 Miniforge / Conda
6   ↓
7 cv environment
8   ↓
9 VS Code + WSL extension
```

Keep course Python environments and projects **inside WSL**.

## macOS / Linux

Recommended workflow:

```
1 macOS / Linux
2   ↓
3 Terminal
4   ↓
5 Miniforge / Conda
6   ↓
7 cv environment
8   ↓
9 VS Code
```

No WSL is needed.

# Before We Create the Environment

Open a terminal and check the tools you already have.

## Git

```
1 git --version
```

## Conda

```
1 conda --version
```

## Existing Conda environments

```
1 conda env list
```

Windows users working in WSL can also check:

```
1 uname -a
```



If `conda` is not installed yet, open the **Python & Conda** tutorial and follow the Miniforge installation section.



# The Course Environment

We use a dedicated Conda environment named:

```
1 cv
```

The environment file is maintained in the course repository:

<https://github.com/ariarobotics/cv/blob/main/code/cv-environment.yml>

It defines the software stack used by the course, including:

- Python
- NumPy / SciPy
- OpenCV
- PyTorch / TorchVision
- Matplotlib
- scikit-image / scikit-learn
- Jupyter and VS Code notebook support

**Do not manually assemble your own version of the environment.**

# Get the Environment File

## Option 1 — Clone the course repository

```
1 cd ~/cv_projects
2 git clone https://github.com/ariarobotics/cv.git
3 cd cv/code
```

You should now see:

```
1 ls
```

including:

```
1 cv-environment.yml
```

## Option 2 — Download only the environment file

```
1 curl -L -o cv-environment.yml \
2 https://raw.githubusercontent.com/ariarobotics/cv/main/code/cv-environment.yml
```

# Create the **cv** Environment

From the directory containing `cv-environment.yml`:

```
1 conda env create -f cv-environment.yml
```

This may take several minutes.

The environment file already specifies the name:

```
1 cv
```

When installation finishes:

```
1 conda activate cv
```

Your terminal prompt should now begin with something similar to:

```
1 (cv)
```

## Important

Whenever you run course Python code, make sure the **cv** environment is active.

# Verify the Environment

With `cv` activated:

```
1 python --version
```

Check which Python is being used:

```
1 which python
```

Then verify the main packages:

```
1 python -c "import numpy; print('NumPy:', numpy.__version__)"
2 python -c "import cv2; print('OpenCV:', cv2.__version__)"
3 python -c "import torch; print('PyTorch:', torch.__version__)"
4 python -c "import torchvision; print('TorchVision:', torchvision.__version__)"
```

If all four commands work, the core environment is ready.

# Configure VS Code

Open the project folder from your terminal:

```
1 code .
```

Windows + WSL users should see **WSL: Ubuntu** in the VS Code status area.

Then:

1. Open the Command Palette: **Ctrl+Shift+P**
2. Search for **Python: Select Interpreter**
3. Select the interpreter associated with **cv**

For notebooks, use **Select Kernel** and choose the same **cv** environment.

Verify from Python:

```
1 import sys
2 print(sys.executable)
```

The path should point into the **cv** Conda environment.



# Our First Computer Vision Program

Create a file named:

```
1 first_cv.py
```

Start by importing the libraries:

```
1 from pathlib import Path
2
3 import cv2
4 import matplotlib.pyplot as plt
5 import numpy as np
```

Create a folder for output files:

```
1 output_dir = Path("outputs")
2 output_dir.mkdir(exist_ok=True)
```

# Create and Save a Simple Image

For today's example, we will create a small synthetic image so everyone has the same input.

```
1 height = 360
2 width = 640
3
4 image = np.full(
5     (height, width, 3),
6     235,
7     dtype=np.uint8,
8 )
9
10 # OpenCV uses BGR color ordering.
11 cv2.rectangle(
12     image, (70, 80), (260, 280),
13     (255, 100, 0), thickness=-1
14 )
15
16 cv2.circle(
17     image, (470, 180), 90,
```

We now have a real image file on disk that we can load with OpenCV.

# Load and Inspect the Image

Load the image:

```
1 image_bgr = cv2.imread(  
2     str(output_dir / "sample_image.png")  
3 )
```

Inspect it:

```
1 print("Shape:", image_bgr.shape)  
2 print("Data type:", image_bgr.dtype)
```

You should see something similar to:

```
1 Shape: (360, 640, 3)  
2 Data type: uint8
```

An image is fundamentally a **NumPy array**.

# Display the Image

OpenCV loads color images in **BGR** order.

Matplotlib expects **RGB**, so convert before displaying:

```
1 image_rgb = cv2.cvtColor(  
2     image_bgr,  
3     cv2.COLOR_BGR2RGB  
4 )  
5  
6 plt.imshow(image_rgb)  
7 plt.axis("off")  
8 plt.show()
```

A useful rule:

```
1 OpenCV processing → usually BGR  
2 Matplotlib display → RGB
```

## Note

Using Matplotlib is also convenient in notebooks and avoids problems that can occur with `cv2.imshow()` in some WSL setups.

# A First Image-Processing Operation

Convert the image to grayscale:

```
1 gray = cv2.cvtColor(  
2     image_bgr,  
3     cv2.COLOR_BGR2GRAY  
4 )
```

Detect edges:

```
1 edges = cv2.Canny(  
2     gray,  
3     100,  
4     200  
5 )
```

Display the result:

```
1 plt.imshow(edges, cmap="gray")  
2 plt.axis("off")  
3 plt.show()
```



# What Just Happened?

In only a few lines, we used several ideas that will appear throughout the course:

- An image is represented as a **NumPy array**
- Image dimensions are described by `.shape`
- Pixel values have a data type such as `uint8`
- OpenCV can **load, save, transform, and analyze images**
- Color-channel conventions matter: **BGR vs. RGB**
- Computer vision pipelines are often sequences of simple operations

```
1 image
2   ↓
3 load
4   ↓
5 represent as an array
6   ↓
7 transform / analyze
8   ↓
9 display or use the result
```

# Common Setup Problems

## conda: command not found

Restart the terminal or reload your shell:

```
1 source ~/.bashrc
```

## Python imports fail

First check:

```
1 conda activate cv
2 which python
```

## VS Code runs a different Python

Use:

```
1 Python: Select Interpreter
```

and select **cv**.

## Windows paths and Linux paths are getting mixed

If you are using WSL, keep the course environment and projects inside the WSL/Linux filesystem.

# After Class

Please go through any tutorials that cover tools you are not yet comfortable with:

<https://ariarobotics.github.io/cv/tutorials/>

In particular, make sure you can:

- Activate the `cv` environment
- Use basic Git commands
- Open a project in VS Code
- Run Python scripts and Jupyter notebooks
- Work with basic NumPy arrays

You do **not** need to memorize every Git, Conda, or NumPy command. The tutorials are there as references throughout the semester.